

Cyber Security Task 1

Vulnerability Assessment Report for a Live Website

By Oliwia Mitura, CIN ID: FIT/MAR26/CS7078

TABLE OF CONTENTS:

1. Executive Summary
2. Detailed Findings
 - Finding 01: Service Version Disclosure (Nmap)
 - Finding 02: Red flags analysis with Zap
 - Finding 03: Critical Absence of Security HTTP Headers (DevTools)
3. Remediation Roadmap
4. Conclusion

1. EXECUTIVE SUMMARY

1. Overview

This report outlines the results of a passive security audit performed on the Altoro Mutual web application (demo.testfire.net). The goal of this assessment was to identify visible security weaknesses that could be exploited by malicious actors to compromise customer data, hijack user sessions, or damage the organization's reputation.

2. Overall Security Posture

The current security posture of Altoro Mutual is rated as **Critical**. Several high-impact vulnerabilities were discovered that indicate the application is running on outdated infrastructure and lacks modern web defense mechanisms. **Immediate remediation is required to prevent data theft and unauthorized access.**

2. DETAILED FINDINGS

1. Finding Name: Service Version Disclosure

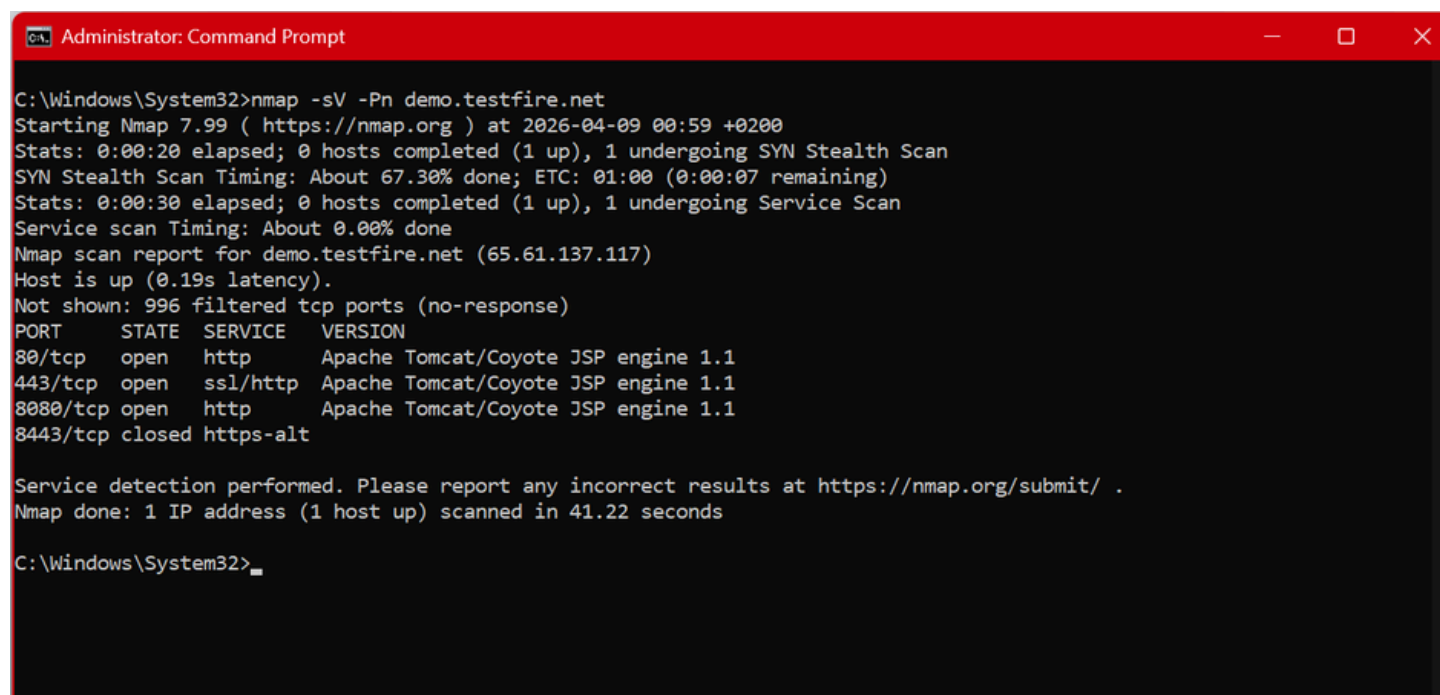
Risk Level: Low

What it is: The server is broadcasting that it runs Apache Tomcat/Coyote 1.1.

Why it matters: It's like a store leaving a sign in the window saying, "We use a 1995 MasterLock on the back door." It tells attackers exactly which tools they need to break in.

The Fix: Configure the server to hide the "Server" header so it only says "Web Server" without the version number.

Below is the output of the Nmap scan conducted on April 9, 2026 at 00:59am (Warsaw local time).

A screenshot of a Windows Command Prompt window titled "Administrator: Command Prompt". The window has a red title bar and standard Windows window controls. The command prompt shows the execution of an Nmap scan on demo.testfire.net. The output includes the Nmap version (7.99), the start time (2026-04-09 00:59 +0200), and the scan progress. The scan results show that the host is up and that three services are running: http on port 80, ssl/http on port 443, and http on port 8080, all using Apache Tomcat/Coyote JSP engine 1.1. The scan was completed in 41.22 seconds.

```
C:\Windows\System32>nmap -sV -Pn demo.testfire.net
Starting Nmap 7.99 ( https://nmap.org ) at 2026-04-09 00:59 +0200
Stats: 0:00:20 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 67.30% done; ETC: 01:00 (0:00:07 remaining)
Stats: 0:00:30 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Nmap scan report for demo.testfire.net (65.61.137.117)
Host is up (0.19s latency).
Not shown: 996 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Apache Tomcat/Coyote JSP engine 1.1
443/tcp   open  ssl/http     Apache Tomcat/Coyote JSP engine 1.1
8080/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
8443/tcp  closed https-alt

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 41.22 seconds

C:\Windows\System32>
```

2. Finding Name: Red flags analysis with Zap

Risk Level: Critical

Below is the output of the Zap scan conducted on April 9, 2026 at 01:06am (Warsaw local time).

Problem 1: SQL Injection

The screenshot shows the ZAP interface with the 'Alerts' tab selected. In the left sidebar, 'SQL Injection (11)' is highlighted. The main panel displays details for an SQL Injection alert:

- URL:** http://demo.testfire.net/doLogin
- Risk:** High
- Confidence:** Medium
- Parameter:** uid
- Attack:** ZAP' OR '1'='1' --
- Evidence:**
- CWE ID:** 89
- WASC ID:** 19
- Source:** Active (40018 - SQL Injection)
- Input Vector:** Form Query
- Description:**

The bottom status bar shows 'Alerts 2 5 7 5' and 'Main Proxy: localhost:8080'.

Red Flag

- **What is it?** This is one of the most dangerous web vulnerabilities. It happens when an attacker can send "commands" directly to your database through a login or search box.
- **The Evidence:** ZAP successfully used the attack ZAP' OR '1'='1' -- on the login page (uid parameter).
- **Why it matters:** An attacker can bypass the login screen without a password, download your entire customer database, or even delete all your records.
- **Remediation (The Fix):** Stop the website from "trusting" user input. Use **Parameterized Queries** (Prepared Statements) so the database treats input as text, not as a command.

Problem 2: Cross-Site Scripting (Reflected)

The screenshot shows the ZAP interface with the 'Alerts' tab selected. In the left sidebar, 'Cross Site Scripting (Reflected) (2)' is highlighted. The main panel displays details for a Cross Site Scripting (Reflected) alert:

- URL:** http://demo.testfire.net/search.jsp?query=%3C%2Fp%3E%3Cscript%3Ealert%281%29%3B%3C%2Fscript%3E%3Cp%3E
- Risk:** High
- Confidence:** Medium
- Parameter:** query
- Attack:** <p><script>alert(1);</script><p>
- Evidence:** <p><script>alert(1);</script><p>
- CWE ID:** 79
- WASC ID:** 8
- Source:** Active (40012 - Cross Site Scripting (Reflected))
- Input Vector:** URL Query String
- Description:**

The bottom status bar shows 'Alerts 2 5 7 5' and 'Main Proxy: localhost:8080'. On the right, a 'Current Status' bar shows various icons and counts.

Red Flag

- **What is it?** The website takes a user's input (like a search term) and "reflects" it back onto the page without checking if it contains malicious code.
- **The Evidence:** ZAP successfully injected a script into the search query: <script>alert(1);</script>.
- **Why it matters:** A hacker can send a specially crafted link to your customers. If they click it, the hacker can steal their login session "cookies" and take over their bank account.
- **Remediation (The Fix):** Use **Output Encoding**. Every piece of data the user types should be cleaned before it is displayed back on the screen.

Problem 3: Missing Anti-Clickjacking Header

The screenshot shows the ZAP Alerts window with the 'Missing Anti-clickjacking Header' alert selected. The alert details are as follows:

Field	Value
URL:	https://demo.testfire.net
Risk:	Medium
Confidence:	Medium
Parameter:	x-frame-options
Attack:	
Evidence:	
CWE ID:	1021
WASC ID:	15
Source:	Passive (10020 - Anti-clickjacking Header)
Alert Reference:	10020-1
Input Vector:	

The left sidebar shows a list of alerts, with 'Missing Anti-clickjacking Header (Systemic)' highlighted. The bottom status bar indicates 'Alerts 2 5 7 5' and 'Main Proxy: localhost:8080'.

Orange Flag

- **What is it?** The website is missing a specific instruction (X-Frame-Options) that tells the browser whether it's okay to put this site inside a "frame" on another website.
- **The Evidence:** ZAP confirmed that the x-frame-options header is missing system-wide.
- **Why it matters:** An attacker could "layer" your bank website under a fake, invisible layer on their own site. A user might think they are clicking a "Claim Prize" button, but they are actually clicking "Transfer Money" on your site.
- **Remediation (The Fix):** Add the X-Frame-Options: DENY or SAMEORIGIN header to your web server configuration.

Problem 4: Absence of Anti-CSRF Tokens

The screenshot shows the ZAP Alerts window with the 'Absence of Anti-CSRF Tokens' alert selected. The alert details are as follows:

Field	Value
URL:	https://demo.testfire.net/login.jsp
Risk:	Medium
Confidence:	Low
Parameter:	
Attack:	
Evidence:	<form action="doLogin" method="post" name="login" id="login" onsubmit="return (confirminput(login));">
CWE ID:	352
WASC ID:	9
Source:	Passive (10202 - Absence of Anti-CSRF Tokens)
Input Vector:	
Description:	

The left sidebar shows a list of alerts, with 'Absence of Anti-CSRF Tokens (6)' highlighted. The bottom status bar indicates 'Alerts 2 5 7 5' and 'Main Proxy: localhost:8080'. The current status is shown as 0 red, 0 green, and 0 yellow flags.

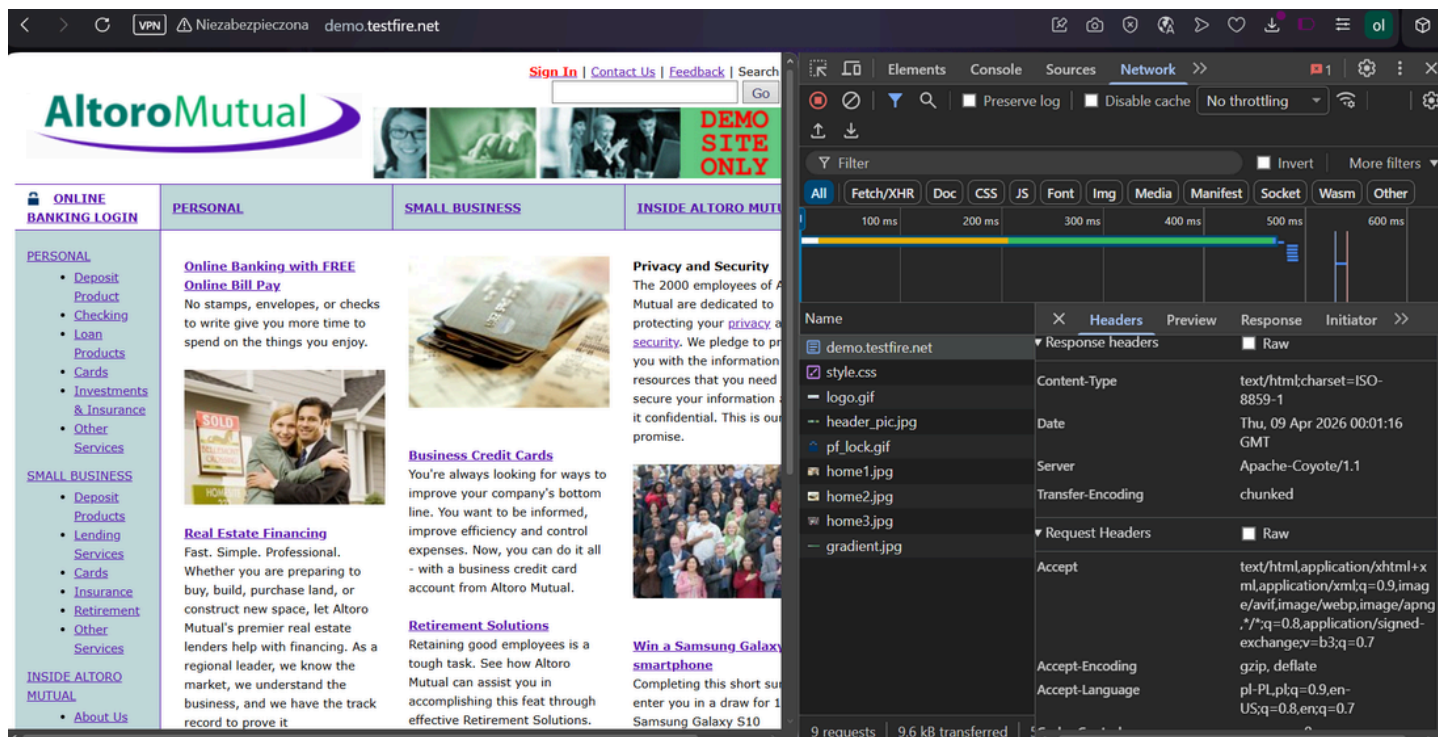
Orange Flag

- **What is it?** The login and transaction forms don't have a "secret handshake" (token) to prove the request is coming from a real user on your site.
- **Why it matters:** An attacker can trick a logged-in user into performing actions they didn't intend to do (like changing their email or password) just by getting them to visit a malicious page.
- **Remediation (The Fix):** Implement **Anti-CSRF Tokens** for every form that changes data or performs a sensitive action.

3. Finding Name: Critical Absence of Security HTTP Headers (DevTools)

Risk Level: Medium

Below is the output of the Network DevTools scan conducted on April 9, 2026 at 01:16am (Warsaw local time).



The Analysis: A manual inspection of the server response (as seen in the provided browser evidence) shows that the web server is failing to send modern security headers. These headers are the primary way a website tells a visitor's browser how to handle data safely.

The Impact: Without X-Frame-Options: This site can be loaded inside an invisible frame on a hacker's website to trick users into clicking buttons (Clickjacking).

- **Without Content-Security-Policy (CSP):** The browser cannot distinguish between a legitimate script from the bank and a malicious script injected by an attacker.
- **Without Strict-Transport-Security (HSTS):** Users could be downgraded to an unencrypted connection, allowing their bank details to be intercepted on public Wi-Fi.

The Fix: The server administrator must update the web.xml or server configuration to include these missing security headers.

3. REMEDIATION ROADMAP

Priority 1: Immediate Action (Fix within 0–7 Days)

These vulnerabilities are the "Open Vault Doors." They are easy to exploit and lead to total data loss.

- **Fix SQL Injection:** Implement **Parameterized Queries** (Prepared Statements) across all database interactions. This ensures user input is never executed as a command.
- **Fix Reflected XSS:** Apply **Context-Aware Output Encoding**. The application must "clean" any data it displays back to the user to ensure it cannot contain malicious scripts.
- **Update Apache Tomcat:** The server is running version 1.1 (dangerously old). **Upgrade to the latest stable version** immediately to patch years of known public vulnerabilities.

Priority 2: Infrastructure Hardening (Fix within 15–30 Days)

These fixes strengthen the "Walls" of the application to prevent more sophisticated attacks.

- **Deploy Security Headers:** Configure the web server to send the following headers with every response:
 - X-Frame-Options: SAMEORIGIN (Stops Clickjacking)
 - Content-Security-Policy (Stops unauthorized script execution)
 - Strict-Transport-Security (Forces secure HTTPS connections)
- **Implement Anti-CSRF Tokens:** Add unique, per-session tokens to all forms (Login, Transfer, Search) to ensure actions are only performed by the intended user.

Priority 3: Operational Security (Fix within 60 Days)

These are "Housekeeping" items that reduce the amount of information an attacker can gather.

- **Suppress Server Information:** Configure the server to hide the Server and X-Powered-By headers. The server should ideally return a generic "Web Server" string or nothing at all.
- **Close Unnecessary Ports:** Use a firewall to close Port 8080 and any other ports not strictly required for the public-facing website.
- **Establish a Scanning Schedule:** Perform a **Passive Vulnerability Assessment** (like this one) at least once a quarter or after every major code update.

4. CONCLUSION

The security assessment of **Altoro Mutual** (demo.testfire.net) has revealed a high-risk security posture characterized by critical software vulnerabilities and missing defensive configurations. The presence of **SQL Injection** and **Cross-Site Scripting (XSS)** indicates that an attacker could easily compromise customer data and bypass primary authentication methods.

Furthermore, the reliance on **outdated server infrastructure** (Apache Tomcat 1.1) exposes the organization to numerous publicly documented exploits. While these issues are significant,

they are remediable through the systematic application of the provided **Remediation Roadmap**.

Security is not a one-time checkbox but a continuous process of improvement. By addressing these findings, Altoro Mutual will not only secure its current assets but also build a resilient foundation against evolving cyber threats.